Redshift

A rover fleet that explores an unknown super-Earth on its own, with no map and no link home

Agentverse 2026, Robotics track
Technical summary

USER AND PROBLEM

User: a planetary mission team sending rovers where nobody can drive them. Mars is up to 22 minutes away by radio; a planet around another star is years away. Every hazard has to be handled by the rover itself, from its own sensors, before anyone on Earth could hear about it.

Need: NASA's Spirit rover was lost in 2009 when its wheels dug into soft soil it could not see. Perseverance carries only a few dozen sample tubes, so every sample is a judgement call. A rover has to notice trouble, back off, and judge what is worth keeping, all by itself.

WHAT IT DOES

- **Perceive:** each rover has a 1008-ray LiDAR (8 m, 5 sweeps/s), a mast camera, an IMU, wheel encoders, motor torque feedback, a belly wetness probe and a spectrometer. It knows nothing else: no map, no ground truth.
- **Map and share:** LiDAR builds one fleet height map of a 120 m sector. Frontiers (the edge of the known) are split between rovers so they spread out.
- **Learn on the fly:** the fleet starts assuming 30 deg climbs. A stall with motors near their limit lowers the shared limit ("slope 21 deg with motors at 87%") and every rover replans.
- **Adapt mid-run:** judges place rockfalls, soft sand, a toppling spire, a dust storm, a motor fault or a herd of creatures anywhere, live. Rovers only find out through their sensors and visibly replan.
- **Science and life:** lakes found from missing LiDAR echoes plus the camera; moving LiDAR clusters tracked as possible life and watched from at least 4 m; samples taken with the right tool into 3 tubes and cached at the lander.
- **Explainable:** every event names the reading that caused it, and the dashboard shows each rover's camera, LiDAR, slip and load next to what it does.

SYSTEM ARCHITECTURE

Sensors (simulated) MuJoCo physics at 2 g: LiDAR ray casts, camera renders, IMU, encoders, motor torques.	**Tier 1: rule brain** Per rover, 5 Hz. Map update, frontiers, anytime A*, state machine, all safety reactions. **Always in control.**	**Tier 2: Gemma (local)** Own thread, never blocks tier 1. Picks among goals tier 1 already checked, reads camera + numbers, parses operator text.
Fleet layer Shared height map, frontier claims, slope limit, soft, water and keep-out zones, sample cache.	**Actuators** Skid-steer wheel torques, instruments (drill, scoop, sniffer, swab).	**Mission control** FastAPI + WebSocket to a React / Three.js dashboard: map, sensors, events, hazards, chat.

Sensors → tier 1 → actuators every 200 ms. Tier 2 suggests; tier 1 checks every suggestion against the map, slope limit and battery before using it.

WHERE EACH MODEL RUNS

Component	Runs on	Budget	Measured
Rule brain (map, A*, decisions), per rover	CPU, Python. Intel Core 5 210H laptop	40 ms per 200 ms cycle	avg 3.3 ms, p95 8.4 ms, max 34 ms; 0 of 10,500 cycles over budget (3 rovers, 700 s, 6 hazards)
Gemma 4 E2B (gemma4:e2b) via Ollama	Local GPU. RTX 4070 laptop	3 s per call, dropped at 4 s	goal choice about 0.3 s, camera + sensor anomaly about 0.4 s (warm)
Camera frames (for Gemma and liquid check)	MuJoCo OpenGL, same machine	throttled, counted as sensor time	320 x 240 for checks
Physics and sensor simulation	CPU, MuJoCo	n/a (stands in for the world)	whole sim 5.6x faster than real time

No cloud service anywhere. Everything runs on one laptop and works with the network cable pulled. With Gemma off or offline the rovers run on rules alone.

CRITICAL FAILURE MODE: STUCK IN SOFT GROUND

Detect: wheels turning but rover hardly moving: slip over 60% for 1.5 s, on gentle ground, with at least two loaded wheels spinning (rules out slopes, turns and a rover hung on a rock).
React: stop, report "wheels spinning but not moving (slip 85%). Soft ground", mark a generous soft zone that the whole fleet avoids, back out along its own ruts, replan. Near known soft ground it trusts weaker evidence. Failsafe: dig out after 30 s.

SAFETY

- **Rules hold authority.** Gemma can only choose from options the rules already judged safe. A bad, slow or missing answer means the rules decide.
- **When unsure, slow down, then ask.** Anything unexpected makes the rover slow down before it replans. If the rover still cannot tell what it sees, it holds still and asks mission control, taking the safe option if nobody answers in time.
- **Always able to get home:** battery for the trip back plus 15% (25% when dust covers the lander's solar panels).
- **Planetary protection:** possible life is watched from 4 m or more and never touched; a rover backs off if a creature approaches.
- **Operator commands are checked** by the same rules, and refused with reasons if unsafe.

LIMITS

- A simulation, not hardware. Position comes from the simulator (stands in for localisation; no SLAM).
- LiDAR hits on creatures and liquid are routed by object id, a stated shortcut for scan-vs-map separation.
- The rovers share one map instantly; real inter-rover links would be patchy.
- The liquid check is a colour test on the camera picture. A 2B-parameter model can misread a scene, which is why it only advises.
- Wear, towing and the camera object detector are built on side branches but not in this demo build.

Simulator: MuJoCo 3D physics, 2 g, 120 m sector, 3 rovers. Stack: Python, FastAPI, React, Three.js, Ollama.

github.com/Stanleyhoo1/Redshift